

✓ From “Selfish” to Optimal:

A Data-Driven Analysis of Shot Decision Quality Using Expected Goals

In football discussions, players are often labeled as “selfish” when they choose to shoot instead of passing. However, such judgments are subjective and outcome-driven.

A missed shot does not necessarily imply a poor decision, and a goal does not always indicate an optimal one.

In this project, we propose a **data-driven framework** to evaluate shot decision quality by comparing:

- the expected goal value of the **actual shot**
- the expected goal value of a **reasonable passing alternative** from the same location under real match context.

Our goal is to figure out can we quantify and compare player-level shooting decision quality using aggregated decision cost derived from 10 seasons of Premier League data.

✓ Data Sources Overview

To study shot decision quality, we use **three independent real-world data sources**, each serving a distinct role.

Data Source 1 — StatsBomb Open Data (Event-level)

Used for:

- Shot events (xG, shooter)
- Pass events
- Player and team identity
- Match time and location

This is our **core event-level dataset**.

Data Source 2 — Football-Data.co.uk (Match Results)

Used for:

- Match results
- Team performance history
- Computing recent form (last 5 matches before each game)

Data Source 3 — League Standings

Used for:

- League position
- Points at the time of the match
- Contextual pressure indicators

Data Integration Strategy

All datasets are joined using:

- season
- match_date
- home_team
- away_team

A small team-name mapping table is used to standardize naming differences.

Study Window

To improve sample size, stabilize zone-based passing value estimation, and support more reliable player-level comparisons, we extend the study window from the most recent 3 seasons to the most recent **10 Premier League seasons**.

```
import pandas as pd
import requests

BASE = "https://raw.githubusercontent.com/statsbomb/open-data/master/data"

def get_json(url):
    return requests.get(url).json()
```

```

# load competitions
comps = pd.DataFrame(get_json(f"{BASE}/competitions.json"))

# keep Premier League only
epl = comps[comps["competition_name"] == "Premier League"].copy()

# sort by season and keep the most recent 10 EPL seasons
# (this is the main change from the original 3-season version)
epl = epl.sort_values("season_name", ascending=False).head(10).reset_index(drop=True)

print("Most recent 10 Premier League seasons available in StatsBomb:")
display(epl[["competition_id", "season_id", "season_name"]])

# choose one season as a demonstration example
row = epl.iloc[0]
competition_id = int(row["competition_id"])
season_id = int(row["season_id"])
season_name = row["season_name"]

print(f"Demo season selected for event preview: {season_name}")

# load matches for the selected season
matches = pd.DataFrame(
    get_json(f"{BASE}/matches/{competition_id}/{season_id}.json")
)

print("Matches shape:", matches.shape)
display(matches[["match_id", "match_date", "home_team", "away_team"]].head())

# load events for one example match
match_id = int(matches.iloc[0]["match_id"])
events = pd.DataFrame(
    get_json(f"{BASE}/events/{match_id}.json")
)

print(f"Loaded events for match_id = {match_id}")
print("Events shape:", events.shape)
display(events.head())

# inspect key columns in a cleaner way
events_preview = pd.DataFrame({
    "id": events["id"],
    "minute": events["minute"],
    "second": events["second"],
    "type_name": events["type"].apply(lambda x: x.get("name") if isinstance(x, dict) else None),
    "possession": events["possession"],
    "possession_team_name": events["possession_team"].apply(lambda x: x.get("name") if isinstance(x, dict) else None),
    "play_pattern_name": events["play_pattern"].apply(lambda x: x.get("name") if isinstance(x, dict) else None),
})

display(events_preview.head(10))

```

Most recent 10 Premier League seasons available in StatsBomb:

	competition_id	season_id	season_name	
0		2	27	2015/2016
1		2	44	2003/2004

Demo season selected for event preview: 2015/2016

Matches shape: (380, 18)

	match_id	match_date	home_team	away_team
0	3754058	2016-01-02	{'home_team_id': 22, 'home_team_name': 'Leices...	{'away_team_id': 28, 'away_team_name': 'AFC Bo...
1	3754245	2015-10-17	{'home_team_id': 27, 'home_team_name': 'West B...	{'away_team_id': 41, 'away_team_name': 'Sunder...
2	3754136	2015-12-19	{'home_team_id': 37, 'home_team_name': 'Newcas...	{'away_team_id': 59, 'away_team_name': 'Aston ...
3	3754037	2016-04-30	{'home_team_id': 29, 'home_team_name': 'Everto...	{'away_team_id': 28, 'away_team_name': 'AFC Bo...
4	3754039	2016-02-13	{'home_team_id': 31, 'home_team_name': 'Crysta...	{'away_team_id': 23, 'away_team_name': 'Watfor...

Loaded events for match_id = 3754058

Events shape: (3576, 35)

	id	index	period	timestamp	minute	second	type	possession	possession_team	play_pattern	...	clea
0	9153e9f4-f69c-4e04-8f64-505592e212cd	1	1	00:00:00.000	0	0	{'id': 35, 'name': 'Starting XI'}	1	{'id': 22, 'name': 'Leicester City'}	{'id': 1, 'name': 'Regular Play'}	...	
1	3fbcf4e7-94d1-485a-be85-fd26a6af0318	2	1	00:00:00.000	0	0	{'id': 35, 'name': 'Starting XI'}	1	{'id': 22, 'name': 'Leicester City'}	{'id': 1, 'name': 'Regular Play'}	...	
2	06a9a4dc-d9c9-40f6-bd89-437ba7fe682d	3	1	00:00:00.000	0	0	{'id': 18, 'name': 'Half Start'}	1	{'id': 22, 'name': 'Leicester City'}	{'id': 1, 'name': 'Regular Play'}	...	
3	100362ee-9311-4187-bd8a-0201d9db2565	4	1	00:00:00.000	0	0	{'id': 18, 'name': 'Half Start'}	1	{'id': 22, 'name': 'Leicester City'}	{'id': 1, 'name': 'Regular Play'}	...	
4	2ca23eea-a984-47e4-8243-8f00880ad1c9	5	1	00:00:01.753	0	1	{'id': 30, 'name': 'Pass'}	2	{'id': 28, 'name': 'AFC Bournemouth'}	{'id': 9, 'name': 'From Kick Off'}	...	

5 rows × 35 columns

	id	minute	second	type_name	possession	possession_team_name	play_pattern_name
0	9153e9f4-f69c-4e04-8f64-505592e212cd	0	0	Starting XI	1	Leicester City	Regular Play
1	3fbcf4e7-94d1-485a-be85-fd26a6af0318	0	0	Starting XI	1	Leicester City	Regular Play
2	06a9a4dc-d9c9-40f6-bd89-437ba7fe682d	0	0	Half Start	1	Leicester City	Regular Play
3	100362ee-9311-4187-bd8a-	0	0	Half Start	1	Leicester City	Regular Play

3 We use football-data.co.uk to obtain:

- match results
- goals for / against
- match dates

This allows us to compute recent form (last 5 matches) for each team.

```
season_codes = [
    "1415", "1516", "1617", "1718", "1819",
    "1920", "2021", "2122", "2223", "2324"
]

results_list = []

for code in season_codes:
    url = f"https://www.football-data.co.uk/mmz4281/{code}/E0.csv"
    df = pd.read_csv(url)
    df["season_code"] = code
    results_list.append(df)

results = pd.concat(results_list, ignore_index=True)

results[["season_code", "Date", "HomeTeam", "AwayTeam", "FTHG", "FTAG", "FTR"]].head()
```

	season_code	Date	HomeTeam	AwayTeam	FTHG	FTAG	FTR	
0	1415	16/08/14	Arsenal	Crystal Palace	2.0	1.0	H	
1	1415	16/08/14	Leicester	Everton	2.0	2.0	D	
2	1415	16/08/14	Man United	Swansea	1.0	2.0	A	
3	1415	16/08/14	QPR	Hull	0.0	1.0	A	
4	1415	16/08/14	Stoke	Aston Villa	0.0	1.0	A	

5 League table data is used to approximate match pressure. We extract team position and points at the time of each match.

```
import pandas as pd
import numpy as np

# Keep only required columns from the combined 10-season results data
results = results[[
    "season_code", "Date", "HomeTeam", "AwayTeam",
    "FTHG", "FTAG", "FTR"
]].copy()

# Standardize date
results["Date"] = pd.to_datetime(results["Date"], dayfirst=True, errors="coerce")

# Drop rows with critical missing values
results = results.dropna(subset=["season_code", "Date", "HomeTeam", "AwayTeam", "FTHG", "FTAG", "FTR"]).copy()

# Ensure goals are numeric
results["FTHG"] = pd.to_numeric(results["FTHG"], errors="coerce")
results["FTAG"] = pd.to_numeric(results["FTAG"], errors="coerce")
results = results.dropna(subset=["FTHG", "FTAG"]).copy()

results["FTHG"] = results["FTHG"].astype(int)
results["FTAG"] = results["FTAG"].astype(int)

# -----
# Helper: compute points for one team in one match
# -----
def compute_points(row, team):
    if team == row["HomeTeam"]:
        if row["FTR"] == "H":
            return 3
        elif row["FTR"] == "D":
            return 1
        else:
            return 0
    elif team == row["AwayTeam"]:
        if row["FTR"] == "A":
            return 3
        elif row["FTR"] == "D":
            return 1
        else:
            return 0
    return 0

# -----
# Build team-by-match cumulative records
# One row = one team after one match
# -----
records = []

for season_code in sorted(results["season_code"].unique()):
    season_df = results[results["season_code"] == season_code].copy()
    teams = sorted(pd.unique(season_df[["HomeTeam", "AwayTeam"]].values.ravel()))

    for team in teams:
        team_games = season_df[
            (season_df["HomeTeam"] == team) | (season_df["AwayTeam"] == team)
        ].sort_values(["Date"]).copy()

        cum_points = 0
        cum_gf = 0
        cum_ga = 0
        games_played = 0

        for _, row in team_games.iterrows():
            is_home = (team == row["HomeTeam"])

            gf = row["FTHG"] if is_home else row["FTAG"]
```

```

ga = row["FTAG"] if is_home else row["FTHG"]
pts = compute_points(row, team)

games_played += 1
cum_points += pts
cum_gf += gf
cum_ga += ga
cum_gd = cum_gf - cum_ga

records.append({
    "season_code": season_code,
    "Date": row["Date"],
    "team": team,
    "opponent": row["AwayTeam"] if is_home else row["HomeTeam"],
    "is_home": is_home,
    "games_played": games_played,
    "match_points": pts,
    "goals_for_match": gf,
    "goals_against_match": ga,
    "cum_points": cum_points,
    "cum_gf": cum_gf,
    "cum_ga": cum_ga,
    "cum_gd": cum_gd,
    "HomeTeam": row["HomeTeam"],
    "AwayTeam": row["AwayTeam"]
})

league_points = pd.DataFrame(records)

print("league_points shape:", league_points.shape)
display(league_points.head(10))

# -----
# Compute league position after each match date
# Ranking rule:
# 1. cumulative points
# 2. cumulative goal difference
# 3. cumulative goals for
# 4. team name (for deterministic ordering only)
# -----
ranking_frames = []

for season_code in sorted(league_points["season_code"].unique()):
    season_lp = league_points[league_points["season_code"] == season_code].copy()

    for current_date in sorted(season_lp["Date"].unique()):
        snapshot = season_lp[season_lp["Date"] <= current_date].copy()

        # keep latest record for each team up to this date
        snapshot = (
            snapshot.sort_values(["team", "Date"])
                    .groupby("team", as_index=False)
                    .tail(1)
                    .copy()
        )

        snapshot = snapshot.sort_values(
            by=["cum_points", "cum_gd", "cum_gf", "team"],
            ascending=[False, False, False, True]
        ).reset_index(drop=True)

        snapshot["league_position"] = range(1, len(snapshot) + 1)
        snapshot["ranking_date"] = current_date

        ranking_frames.append(snapshot[[
            "season_code", "ranking_date", "team",
            "cum_points", "cum_gd", "cum_gf", "league_position"
        ]])

league_table_daily = pd.concat(ranking_frames, ignore_index=True)

print("league_table_daily shape:", league_table_daily.shape)
display(league_table_daily.head(20))

```

```

/tmp/ipykernel_7738/3470178259.py:11: UserWarning: Could not infer format, so each element will be parsed individually, falling back to
results["Date"] = pd.to_datetime(results["Date"], dayfirst=True, errors="coerce")
league_points shape: (7600, 15)

```

	season_code	Date	team	opponent	is_home	games_played	match_points	goals_for_match	goals_against_match	cum
0	1415	2014-08-16	Arsenal	Crystal Palace	True	1	3	2		1
1	1415	2014-08-23	Arsenal	Everton	False	2	1	2		2
2	1415	2014-08-31	Arsenal	Leicester	False	3	1	1		1
3	1415	2014-09-13	Arsenal	Man City	True	4	1	2		2
4	1415	2014-09-20	Arsenal	Aston Villa	False	5	3	3		0
5	1415	2014-09-27	Arsenal	Tottenham	True	6	1	1		1
6	1415	2014-10-05	Arsenal	Chelsea	False	7	0	0		2
7	1415	2014-10-18	Arsenal	Hull	True	8	1	2		2
8	1415	2014-10-25	Arsenal	Sunderland	False	9	3	2		0
9	1415	2014-11-01	Arsenal	Burnley	True	10	3	3		0

```

league_table_daily shape: (22303, 7)

```

	season_code	ranking_date	team	cum_points	cum_gd	cum_gf	league_position
0	1415	2014-08-16	Arsenal	3	1	2	1
1	1415	2014-08-16	Swansea	3	1	2	2
2	1415	2014-08-16	Aston Villa	3	1	1	3
3	1415	2014-08-16	Hull	3	1	1	4
4	1415	2014-08-16	Tottenham	3	1	1	5
5	1415	2014-08-16	Everton	1	0	2	6
6	1415	2014-08-16	Leicester	1	0	2	7
7	1415	2014-08-16	Sunderland	1	0	2	8
8	1415	2014-08-16	West Brom	1	0	2	9
9	1415	2014-08-16	Crystal Palace	0	-1	1	10
10	1415	2014-08-16	Man United	0	-1	1	11
11	1415	2014-08-16	QPR	0	-1	0	12
12	1415	2014-08-16	Stoke	0	-1	0	13
13	1415	2014-08-16	West Ham	0	-1	0	14

7 Join Keys

We align all datasets using:

- season
- match_date
- home_team
- away_team

Final structure: Shot Event → Match → Team → Recent Form → League Position

Join key part haven't done yet

9 Method Summary

For each shot:

1. Obtain xG_{shot} from StatsBomb
2. Estimate xG_{pass} from historical pass-to-shot patterns
3. Compute: $\Delta = xG_{pass} - xG_{shot}$

4. Introduce context weight x:

- match time
- score difference
- league position
- recent form

Decision Cost = $\Delta \times x$

We then aggregate Decision Cost at player level to evaluate systematic decision-making quality.

```
# -----
# Cell 7: Profile the 10-season results data
# -----
print("results shape:", results.shape)
print("\nColumns:")
print(results.columns.tolist())

print("\nDtypes:")
display(results.dtypes)

print("\nMissing values:")
display(results.isna().sum().sort_values(ascending=False).to_frame("missing_count"))

print("\nDistinct teams across 10 seasons:")
all_teams = pd.unique(results[["HomeTeam", "AwayTeam"]].values.ravel())
print("Number of unique team names:", len(all_teams))
display(pd.Series(sorted(all_teams), name="team").head(40))

print("\nSeason counts:")
display(results["season_code"].value_counts().sort_index().to_frame("num_matches"))

print("\nBasic score summary:")
display(results[["FTHG", "FTAG"]].describe())
```


results shape: (3800, 7)

Columns:

['season_code', 'Date', 'HomeTeam', 'AwayTeam', 'FTHG', 'FTAG', 'FTR']

Dtypes:

	0
season_code	object
Date	datetime64[ns]
HomeTeam	object
AwayTeam	object
FTHG	int64
FTAG	int64
FTR	object

dtype: object

Missing values:

	missing_count	
season_code	0	
Date	0	
HomeTeam	0	
AwayTeam	0	
FTHG	0	
FTAG	0	
FTR	0	

Distinct teams across 10 seasons:

Number of unique team names: 34

	team
0	Arsenal
1	Aston Villa
2	Bournemouth
3	Brentford
4	Brighton
5	Burnley
6	Cardiff
7	Chelsea
8	Crystal Palace

```
# -----
# Cell 8: Load all available EPL matches from StatsBomb open data
# -----
statsbomb_matches_list = []

for _, row in epl.iterrows():
    competition_id = int(row["competition_id"])
    season_id = int(row["season_id"])
    season_name = row["season_name"]

    try:
        season_matches = pd.DataFrame(
            get_json(f"{BASE}/matches/{competition_id}/{season_id}.json")
        )
        season_matches["competition_id"] = competition_id
        season_matches["season_id"] = season_id
        season_matches["season_name"] = season_name
        statsbomb_matches_list.append(season_matches)
        print(f"[OK] {season_name}: {season_matches.shape}")
    except Exception as e:
        print(f"[FAILED] {season_name}: {e}")

matches_all = pd.concat(statsbomb_matches_list, ignore_index=True)
print("\nmatches_all shape:", matches_all.shape)
display(matches_all.head())

24 30 Sheffield United
```

25 Southampton
[OK] 2015/2016: (380, 21)
[OK] 2003/2004: (38, 21)
26 Stoke

27 Chelsea Sunderland (418, 21)

28	match_id	match_date	kick_off	competition	season	home_team	away_team	home_score	aw
0	3754058	2016-01-02	16:00:00.000	{'competition_id': 2, 'country_name': 'England...'	{'season_id': 27, 'season_name': '2015/2016'}	{'home_team_id': 22, 'home_team_name': 'Leices...'	{'away_team_id': 28, 'away_team_name': 'AFC Bo...'	0	
32	West Ham			{'competition_id': 2, 'country_name': 'England...'	{'season_id': 27, 'season_name': '2015/2016'}	{'home_team_id': 27, 'home_team_name': 'West B...'	{'away_team_id': 41, 'away_team_name': 'Sunder...'	1	
33	3754245	2015-10-17	16:00:00.000	{'competition_id': 2, 'country_name': 'England...'	{'season_id': 27, 'season_name': '2015/2016'}	{'home_team_id': 27, 'home_team_name': 'West B...'	{'away_team_id': 41, 'away_team_name': 'Sunder...'	1	
dtype: object									
2	3754136	2015-12-19	18:30:00.000	{'competition_id': 2, 'country_name': 'England...'	{'season_id': 27, 'season_name': '2015/2016'}	{'home_team_id': 37, 'home_team_name': 'Newcas...'	{'away_team_id': 59, 'away_team_name': 'Aston ...'	1	
1415 380									
3	3754037	2016-04-30	16:00:00.000	{'competition_id': 2, 'country_name': 'England...'	{'season_id': 27, 'season_name': '2015/2016'}	{'home_team_id': 29, 'home_team_name': 'Everto...'	{'away_team_id': 28, 'away_team_name': 'AFC Bo...'	2	
1516 380									
1617 380									
4	3754039	2016-02-13	16:00:00.000	{'competition_id': 2, 'country_name': 'England...'	{'season_id': 27, 'season_name': '2015/2016'}	{'home_team_id': 31, 'home_team_name': 'Crysta...'	{'away_team_id': 23, 'away_team_name': 'Watfor...'	1	
5 rows x 10 columns									
2122 380									
2223 380									

```
# -----
# Cell 9: Flatten match-level metadata
# -----
def extract_name(x, key="name"):
    if isinstance(x, dict):
        return x.get(key)
    return None

matches_flat = matches_all.copy()

matches_flat["match_date"] = pd.to_datetime(matches_flat["match_date"], errors="coerce")

matches_flat["home_team_name"] = matches_flat["home_team"].apply(lambda x: x.get("home_team_name") if isinstance(x, dict) else
matches_flat["away_team_name"] = matches_flat["away_team"].apply(lambda x: x.get("away_team_name") if isinstance(x, dict) else

matches_flat["home_team_name"] = matches_flat["home_team_name"].replace({
    "Manchester City": "Man City",
    "Manchester United": "Man United",
    "Tottenham Hotspur": "Tottenham",
    "Wolverhampton Wanderers": "Wolves",
    "Leicester City": "Leicester",
    "Newcastle United": "Newcastle",
    "West Bromwich Albion": "West Brom",
    "Norwich City": "Norwich",
    "Nottingham Forest": "Nott'm Forest"
})

matches_flat["away_team_name"] = matches_flat["away_team_name"].replace({
    "Manchester City": "Man City",
    "Manchester United": "Man United",
    "Tottenham Hotspur": "Tottenham",
    "Wolverhampton Wanderers": "Wolves",
    "Leicester City": "Leicester",
    "Newcastle United": "Newcastle",
    "West Bromwich Albion": "West Brom",
    "Norwich City": "Norwich",
    "Nottingham Forest": "Nott'm Forest"
})

matches_flat = matches_flat[[
    "match_id", "match_date", "season_name",
    "home_team_name", "away_team_name"
]].drop_duplicates().copy()
```

```
print("matches_flat shape:", matches_flat.shape)
display(matches_flat.head())
```

matches_flat shape: (418, 5)

	match_id	match_date	season_name	home_team_name	away_team_name
0	3754058	2016-01-02	2015/2016	Leicester	AFC Bournemouth
1	3754245	2015-10-17	2015/2016	West Brom	Sunderland
2	3754136	2015-12-19	2015/2016	Newcastle	Aston Villa
3	3754037	2016-04-30	2015/2016	Everton	AFC Bournemouth
4	3754039	2016-02-13	2015/2016	Crystal Palace	Watford

```
# -----
# Cell 10: Load all available EPL event data from StatsBomb
# -----
statsbomb_events_list = []

match_ids = matches_flat["match_id"].dropna().astype(int).unique().tolist()

for i, match_id in enumerate(match_ids):
    try:
        ev = pd.DataFrame(get_json(f"{BASE}/events/{match_id}.json"))
        ev["match_id"] = match_id
        statsbomb_events_list.append(ev)
        if (i + 1) % 50 == 0:
            print(f"Loaded {i + 1}/{len(match_ids)} matches")
    except Exception as e:
        print(f"[FAILED] match_id={match_id}: {e}")

events_all = pd.concat(statsbomb_events_list, ignore_index=True)
print("\nevents_all shape:", events_all.shape)
display(events_all.head())
```

Loaded 50/418 matches
 Loaded 100/418 matches
 Loaded 150/418 matches
 Loaded 200/418 matches
 Loaded 250/418 matches
 Loaded 300/418 matches
 Loaded 350/418 matches
 Loaded 400/418 matches

events_all shape: (1443184, 42)

	id	index	period	timestamp	minute	second	type	possession	possession_team	play_pattern	...	ball
0	9153e9f4-f69c-4e04-8f64-505592e212cd	1	1	00:00:00.000	0	0	{'id': 35, 'name': 'Starting XI'}	1	{'id': 22, 'name': 'Leicester City'}	{'id': 1, 'name': 'Regular Play'}	...	
1	3fbcf4e7-94d1-485a-be85-fd26a6af0318	2	1	00:00:00.000	0	0	{'id': 35, 'name': 'Starting XI'}	1	{'id': 22, 'name': 'Leicester City'}	{'id': 1, 'name': 'Regular Play'}	...	
2	06a9a4dc-d9c9-40f6-bd89-437ba7fe682d	3	1	00:00:00.000	0	0	{'id': 18, 'name': 'Half Start'}	1	{'id': 22, 'name': 'Leicester City'}	{'id': 1, 'name': 'Regular Play'}	...	
3	100362ee-9311-4187-bd8a-0201d9db2565	4	1	00:00:00.000	0	0	{'id': 18, 'name': 'Half Start'}	1	{'id': 22, 'name': 'Leicester City'}	{'id': 1, 'name': 'Regular Play'}	...	
4	2ca23eea-a984-47e4-8243-8f00880ad1c9	5	1	00:00:01.753	0	1	{'id': 30, 'name': 'Pass'}	2	{'id': 28, 'name': 'AFC Bournemouth'}	{'id': 9, 'name': 'From Kick Off'}	...	

5 rows × 42 columns

```
# -----
# Cell 10: Load all available EPL event data from StatsBomb
# -----
statsbomb_events_list = []

match_ids = matches_flat["match_id"].dropna().astype(int).unique().tolist()

for i, match_id in enumerate(match_ids):
```

```

try:
    ev = pd.DataFrame(get_json(f"{BASE}/events/{match_id}.json"))
    ev["match_id"] = match_id
    statsbomb_events_list.append(ev)
    if (i + 1) % 50 == 0:
        print(f"Loaded {i + 1}/{len(match_ids)} matches")
except Exception as e:
    print(f"[FAILED] match_id={match_id}: {e}")

events_all = pd.concat(statsbomb_events_list, ignore_index=True)
print("\nevents_all shape:", events_all.shape)
display(events_all.head())

```

Loaded 50/418 matches
 Loaded 100/418 matches
 Loaded 150/418 matches
 Loaded 200/418 matches
 Loaded 250/418 matches
 Loaded 300/418 matches
 Loaded 350/418 matches
 Loaded 400/418 matches

events_all shape: (1443184, 42)

	id	index	period	timestamp	minute	second	type	possession	possession_team	play_pattern	...	ball
0	9153e9f4-f69c-4e04-8f64-505592e212cd	1	1	00:00:00.000	0	0	{'id': 35, 'name': 'Starting XI'}	1	{'id': 22, 'name': 'Leicester City'}	{'id': 1, 'name': 'Regular Play'}	...	
1	3fbcf4e7-94d1-485a-be85-fd26a6af0318	2	1	00:00:00.000	0	0	{'id': 35, 'name': 'Starting XI'}	1	{'id': 22, 'name': 'Leicester City'}	{'id': 1, 'name': 'Regular Play'}	...	
2	06a9a4dc-d9c9-40f6-bd89-437ba7fe682d	3	1	00:00:00.000	0	0	{'id': 18, 'name': 'Half Start'}	1	{'id': 22, 'name': 'Leicester City'}	{'id': 1, 'name': 'Regular Play'}	...	
3	100362ee-9311-4187-bd8a-0201d9db2565	4	1	00:00:00.000	0	0	{'id': 18, 'name': 'Half Start'}	1	{'id': 22, 'name': 'Leicester City'}	{'id': 1, 'name': 'Regular Play'}	...	
4	2ca23eea-a984-47e4-8243-8f00880ad1c9	5	1	00:00:01.753	0	1	{'id': 30, 'name': 'Pass'}	2	{'id': 28, 'name': 'AFC Bournemouth'}	{'id': 9, 'name': 'From Kick Off'}	...	

5 rows × 42 columns

```

# -----
# Cell 11: Flatten core event fields
# -----
def dict_get(x, key):
    if isinstance(x, dict):
        return x.get(key)
    return None

events_flat = events_all.copy()

events_flat["type_name"] = events_flat["type"].apply(lambda x: dict_get(x, "name"))
events_flat["possession_team_name"] = events_flat["possession_team"].apply(lambda x: dict_get(x, "name"))
events_flat["play_pattern_name"] = events_flat["play_pattern"].apply(lambda x: dict_get(x, "name"))
events_flat["team_name"] = events_flat["team"].apply(lambda x: dict_get(x, "name"))
events_flat["player_name"] = events_flat["player"].apply(lambda x: dict_get(x, "name"))

# location
events_flat["x"] = events_flat["location"].apply(lambda x: x[0] if isinstance(x, list) and len(x) >= 2 else np.nan)
events_flat["y"] = events_flat["location"].apply(lambda x: x[1] if isinstance(x, list) and len(x) >= 2 else np.nan)

# pass fields
events_flat["pass_recipient_name"] = events_flat["pass"].apply(lambda x: dict_get(dict_get(x, "recipient"), "name") if isinstance(x, dict) else np.nan)
events_flat["pass_end_x"] = events_flat["pass"].apply(lambda x: x.get("end_location", [np.nan, np.nan])[0] if isinstance(x, dict) else np.nan)
events_flat["pass_end_y"] = events_flat["pass"].apply(lambda x: x.get("end_location", [np.nan, np.nan])[1] if isinstance(x, dict) else np.nan)

# shot fields
events_flat["shot_xg"] = events_flat["shot"].apply(lambda x: x.get("statsbomb_xg") if isinstance(x, dict) else np.nan)
events_flat["shot_outcome_name"] = events_flat["shot"].apply(lambda x: dict_get(dict_get(x, "outcome"), "name") if isinstance(x, dict) else np.nan)

events_flat = events_flat.merge(
    matches_flat,
    on="match_id",

```

```

        how="left"
    )

print("events_flat shape:", events_flat.shape)
display(events_flat[[
    "match_id", "minute", "second", "type_name",
    "team_name", "player_name", "x", "y",
    "possession", "possession_team_name", "shot_xg", "shot_outcome_name"
]].head(10))

```

events_flat shape: (1443184, 58)

	match_id	minute	second	type_name	team_name	player_name	x	y	possession	possession_team_name	shot_xg	shot_outcome_name
0	3754058	0	0	Starting XI	Leicester City	None	NaN	NaN	1	Leicester City	NaN	
1	3754058	0	0	Starting XI	AFC Bournemouth	None	NaN	NaN	1	Leicester City	NaN	
2	3754058	0	0	Half Start	AFC Bournemouth	None	NaN	NaN	1	Leicester City	NaN	
3	3754058	0	0	Half Start	Leicester City	None	NaN	NaN	1	Leicester City	NaN	
4	3754058	0	1	Pass	AFC Bournemouth	Dan Gosling	61.0	40.1	2	AFC Bournemouth	NaN	
5	3754058	0	2	Ball Receipt*	AFC Bournemouth	Joshua King	60.4	43.6	2	AFC Bournemouth	NaN	
6	3754058	0	2	Carry	AFC Bournemouth	Joshua King	60.4	43.6	2	AFC Bournemouth	NaN	

```

# -----
# Cell 12: Extract shot and pass tables
# -----
shots = events_flat[events_flat["type_name"] == "Shot"].copy()
passes = events_flat[events_flat["type_name"] == "Pass"].copy()

# keep useful columns
shots = shots[[
    "match_id", "season_name", "match_date",
    "home_team_name", "away_team_name",
    "index", "minute", "second",
    "team_name", "player_name",
    "possession", "possession_team_name",
    "x", "y", "shot_xg", "shot_outcome_name"
]].copy()

passes = passes[[
    "match_id", "season_name", "match_date",
    "home_team_name", "away_team_name",
    "index", "minute", "second",
    "team_name", "player_name",
    "possession", "possession_team_name",
    "x", "y", "pass_end_x", "pass_end_y", "pass_recipient_name"
]].copy()

# basic cleaning
shots = shots.dropna(subset=["x", "y", "shot_xg", "team_name", "player_name"]).copy()
passes = passes.dropna(subset=["x", "y", "team_name", "player_name"]).copy()

print("shots shape:", shots.shape)
print("passes shape:", passes.shape)

display(shots.head())
display(passes.head())

```

shots shape: (10837, 16)
passes shape: (404785, 17)

	match_id	season_name	match_date	home_team_name	away_team_name	index	minute	second	team_name	player_name
92	3754058	2015/2016	2016-01-02	Leicester	AFC Bournemouth	93	1	33	AFC Bournemouth	Junior Stanislas
223	3754058	2015/2016	2016-01-02	Leicester	AFC Bournemouth	224	4	23	AFC Bournemouth	Joshua King
439	3754058	2015/2016	2016-01-02	Leicester	AFC Bournemouth	440	9	20	Leicester City	Riyad Mahrez
578	3754058	2015/2016	2016-01-02	Leicester	AFC Bournemouth	579	13	11	AFC Bournemouth	Joshua King
1169	3754058	2015/2016	2016-01-02	Leicester	AFC Bournemouth	1170	25	47	Leicester City	José Leonardo Ulloa

	match_id	season_name	match_date	home_team_name	away_team_name	index	minute	second	team_name	player_name
4	3754058	2015/2016	2016-01-02	Leicester	AFC Bournemouth	5	0	1	AFC Bournemouth	Dan Gosling
7	3754058	2015/2016	2016-01-02	Leicester	AFC Bournemouth	8	0	2	AFC Bournemouth	Joshua King
9	3754058	2015/2016	2016-01-02	Leicester	AFC Bournemouth	10	0	3	AFC Bournemouth	Andrew Surman
12	3754058	2015/2016	2016-01-02	Leicester	AFC Bournemouth	13	0	6	AFC Bournemouth	Adam Smith
15	3754058	2015/2016	2016-01-02	Leicester	AFC Bournemouth	16	0	9	AFC Bournemouth	Simon Francis

```
# -----  
# Cell 13: Create pitch zones  
# StatsBomb pitch ~ 120 x 80  
# -----  
X_BINS = [0, 30, 60, 90, 120]  
Y_BINS = [0, 20, 40, 60, 80]  
  
def assign_zone(x, y):  
    if pd.isna(x) or pd.isna(y):  
        return np.nan  
  
    x_bin = pd.cut([x], bins=X_BINS, labels=False, include_lowest=True)[0]  
    y_bin = pd.cut([y], bins=Y_BINS, labels=False, include_lowest=True)[0]  
  
    if pd.isna(x_bin) or pd.isna(y_bin):  
        return np.nan  
  
    return f"Z{int(x_bin)}_{int(y_bin)}"  
  
shots["zone"] = shots.apply(lambda r: assign_zone(r["x"], r["y"]), axis=1)  
passes["zone"] = passes.apply(lambda r: assign_zone(r["x"], r["y"]), axis=1)  
  
display(shots[["x", "y", "zone"]].head())  
display(passes[["x", "y", "zone"]].head())
```

	x	y	zone
92	107.1	26.6	Z3_1
223	113.7	54.0	Z3_2
439	115.7	50.5	Z3_2
578	113.9	38.6	Z3_1
1169	96.7	47.2	Z3_2

	x	y	zone
4	61.0	40.1	Z2_2
7	60.4	43.6	Z2_2
9	48.0	41.7	Z1_2
12	37.5	74.6	Z1_3
15	27.4	63.9	Z0_3

```

# -----
# Cell 14: Estimate xG_pass by zone
# For each pass, find the next shot in the same match, same possession, same team
# -----
pass_shot_links = []

# prepare a faster shot lookup
shots_lookup = shots[[
    "match_id", "team_name", "possession", "index", "shot_xg"
]].copy()

for _, p in passes.iterrows():
    candidate_shots = shots_lookup[
        (shots_lookup["match_id"] == p["match_id"]) &
        (shots_lookup["team_name"] == p["team_name"]) &
        (shots_lookup["possession"] == p["possession"]) &
        (shots_lookup["index"] > p["index"])
    ].sort_values("index")

    if len(candidate_shots) > 0:
        next_shot_xg = candidate_shots.iloc[0]["shot_xg"]
        pass_shot_links.append({
            "match_id": p["match_id"],
            "team_name": p["team_name"],
            "possession": p["possession"],
            "pass_index": p["index"],
            "zone": p["zone"],
            "next_shot_xg": next_shot_xg
        })

pass_shot_df = pd.DataFrame(pass_shot_links)

print("pass_shot_df shape:", pass_shot_df.shape)
display(pass_shot_df.head())

zone_xg_pass = (
    pass_shot_df
    .groupby("zone", as_index=False)
    .agg(
        xG_pass=("next_shot_xg", "mean"),
        zone_pass_shot_samples=("next_shot_xg", "size")
    )
)

print("zone_xg_pass shape:", zone_xg_pass.shape)
display(zone_xg_pass.sort_values("zone").head(20))

```

pass_shot_df shape: (52180, 6)

	match_id	team_name	possession	pass_index	zone	next_shot_xg
0	3754058	AFC Bournemouth	4	63	Z1_1	0.020838
1	3754058	AFC Bournemouth	4	65	Z1_1	0.020838
2	3754058	AFC Bournemouth	4	68	Z0_1	0.020838
3	3754058	AFC Bournemouth	4	72	Z1_2	0.020838
4	3754058	AFC Bournemouth	4	75	Z0_2	0.020838



zone_xg_pass shape: (16, 3)

	zone	xG_pass	zone_pass_shot_samples
0	Z0_0	0.110314	595
1	Z0_1	0.097490	1434
2	Z0_2	0.101285	1456
3	Z0_3	0.095918	504
4	Z1_0	0.090639	3076
5	Z1_1	0.095202	3580
6	Z1_2	0.089159	3438
7	Z1_3	0.093408	2898
8	Z2_0	0.084010	5118
9	Z2_1	0.085769	5211
10	Z2_2	0.088118	4986
11	Z2_3	0.082583	5110
12	Z3_0	0.084153	4660
13	Z3_1	0.105515	2813
14	Z3_2	0.111473	2596
15	Z3_3	0.085257	4705

```
# -----
# Cell 15: Attach xG_pass and compute delta
# -----
shots_enriched = shots.merge(
    zone_xg_pass,
    on="zone",
    how="left"
)

shots_enriched["delta"] = shots_enriched["xG_pass"] - shots_enriched["shot_xg"]

print("shots_enriched shape:", shots_enriched.shape)
display(shots_enriched[[
    "match_id", "player_name", "team_name", "zone",
    "shot_xg", "xG_pass", "zone_pass_shot_samples", "delta"
]].head(10))
```

shots_enriched shape: (10837, 20)

	match_id	player_name	team_name	zone	shot_xg	xG_pass	zone_pass_shot_samples	delta
0	3754058	Junior Stanislas	AFC Bournemouth	Z3_1	0.020838	0.105515	2813.0	0.084677
1	3754058	Joshua King	AFC Bournemouth	Z3_2	0.034462	0.111473	2596.0	0.077012
2	3754058	Riyad Mahrez	Leicester City	Z3_2	0.035481	0.111473	2596.0	0.075992
3	3754058	Joshua King	AFC Bournemouth	Z3_1	0.383711	0.105515	2813.0	-0.278196
4	3754058	José Leonardo Ulloa	Leicester City	Z3_2	0.036740	0.111473	2596.0	0.074733
5	3754058	Jamie Vardy	Leicester City	Z3_1	0.575428	0.105515	2813.0	-0.469912
6	3754058	Junior Stanislas	AFC Bournemouth	Z3_1	0.095998	0.105515	2813.0	0.009517
7	3754058	José Leonardo Ulloa	Leicester City	Z3_1	0.047259	0.105515	2813.0	0.058256
8	3754058	Dan Gosling	AFC Bournemouth	Z3_2	0.256448	0.111473	2596.0	-0.144975
9	3754058	Jamie Vardy	Leicester City	Z3_1	0.111973	0.105515	2813.0	-0.006458



```
# -----
# Cell 16: Build team-match table from results for recent form
```

```
# -----
team_match_records = []

for _, row in results.iterrows():
    # home side
    if row["FTR"] == "H":
        home_pts, away_pts = 3, 0
    elif row["FTR"] == "A":
        home_pts, away_pts = 0, 3
    else:
        home_pts, away_pts = 1, 1

    team_match_records.append({
        "season_code": row["season_code"],
        "Date": row["Date"],
        "team": row["HomeTeam"],
        "opponent": row["AwayTeam"],
        "is_home": 1,
        "goals_for": row["FTHG"],
        "goals_against": row["FTAG"],
        "match_points": home_pts
    })

    team_match_records.append({
        "season_code": row["season_code"],
        "Date": row["Date"],
        "team": row["AwayTeam"],
        "opponent": row["HomeTeam"],
        "is_home": 0,
        "goals_for": row["FTAG"],
        "goals_against": row["FTHG"],
        "match_points": away_pts
    })

team_matches = pd.DataFrame(team_match_records)
team_matches = team_matches.sort_values(["season_code", "team", "Date"]).copy()

# recent form = points in previous 5 matches
team_matches["recent_form_5"] = (
    team_matches.groupby(["season_code", "team"])["match_points"]
    .transform(lambda s: s.shift(1).rolling(5, min_periods=1).sum())
)

print("team_matches shape:", team_matches.shape)
display(team_matches.head(10))
```

team_matches shape: (7600, 9)

	season_code	Date	team	opponent	is_home	goals_for	goals_against	match_points	recent_form_5
0	1415	2014-08-16	Arsenal	Crystal Palace	1	2	1	3	NaN
27	1415	2014-08-23	Arsenal	Everton	0	2	2	1	3.0
57	1415	2014-08-31	Arsenal	Leicester	0	1	1	1	4.0
60	1415	2014-09-13	Arsenal	Man City	1	2	2	1	5.0
81	1415	2014-09-20	Arsenal	Aston Villa	0	3	0	3	6.0
100	1415	2014-09-27	Arsenal	Tottenham	1	1	1	1	9.0
133	1415	2014-10-05	Arsenal	Chelsea	0	0	2	0	7.0
140	1415	2014-10-18	Arsenal	Hull	1	2	2	1	6.0
165	1415	2014-10-25	Arsenal	Sunderland	0	2	0	3	6.0
180	1415	2014-11-01	Arsenal	Burnley	1	3	0	3	8.0

```
# -----
# Cell 17: Join recent form and league position by season/team groups
# -----
def season_name_to_code(season_name):
    if pd.isna(season_name):
        return np.nan
    season_name = str(season_name)
    if "/" in season_name:
        start_year, end_year = season_name.split("/")
        return start_year[-2:] + end_year[-2:]
    return np.nan

team_name_map = {
    "Leicester City": "Leicester",
    "Manchester City": "Man City",
    "Manchester United": "Man United",
```

```

    "Tottenham Hotspur": "Tottenham",
    "Wolverhampton Wanderers": "Wolves",
    "Newcastle United": "Newcastle",
    "West Bromwich Albion": "West Brom",
    "Norwich City": "Norwich",
    "Nottingham Forest": "Nott'm Forest"
}

# -----
# Prepare left table: shots_enriched
# -----
shots_enriched = shots_enriched.copy()
shots_enriched["season_code"] = shots_enriched["season_name"].apply(season_name_to_code)
shots_enriched["team_join"] = shots_enriched["team_name"].replace(team_name_map)
shots_enriched["match_date"] = pd.to_datetime(shots_enriched["match_date"], errors="coerce")

shots_enriched = shots_enriched.dropna(subset=["season_code", "team_join", "match_date"]).copy()

# -----
# Prepare recent form table
# -----
form_join = team_matches[["season_code", "Date", "team", "recent_form_5']].copy()
form_join = form_join.rename(columns={"team": "team_join"})
form_join["Date"] = pd.to_datetime(form_join["Date"], errors="coerce")
form_join = form_join.dropna(subset=["season_code", "team_join", "Date"]).copy()

# -----
# Prepare standings table
# -----
standings_join = league_table_daily.copy()
standings_join = standings_join.rename(columns={
    "ranking_date": "Date",
    "team": "team_join"
})
standings_join["Date"] = pd.to_datetime(standings_join["Date"], errors="coerce")
standings_join = standings_join.dropna(subset=["season_code", "team_join", "Date"]).copy()

# -----
# Helper: groupwise merge_asof
# -----
def grouped_asof_merge(left_df, right_df, left_time_col, right_time_col, by_cols, right_keep_cols,
                        direction="backward", allow_exact_matches=False):
    merged_parts = []

    left_df = left_df.copy()
    right_df = right_df.copy()

    # group keys present on both sides
    left_keys = left_df[by_cols].drop_duplicates()
    right_keys = right_df[by_cols].drop_duplicates()
    common_keys = left_keys.merge(right_keys, on=by_cols, how="inner")

    # groups with matching right-side info
    for _, key_row in common_keys.iterrows():
        left_mask = pd.Series(True, index=left_df.index)
        right_mask = pd.Series(True, index=right_df.index)

        for c in by_cols:
            left_mask &= (left_df[c] == key_row[c])
            right_mask &= (right_df[c] == key_row[c])

        left_sub = left_df[left_mask].sort_values(left_time_col).reset_index(drop=True)
        right_sub = right_df[right_mask].sort_values(right_time_col).reset_index(drop=True)

        if len(left_sub) == 0:
            continue

        merged = pd.merge_asof(
            left_sub,
            right_sub[by_cols + [right_time_col] + right_keep_cols],
            left_on=left_time_col,
            right_on=right_time_col,
            direction=direction,
            allow_exact_matches=allow_exact_matches
        )
        merged_parts.append(merged)

    # groups with no matching right-side info: keep left rows and fill nulls
    unmatched_keys = left_keys.merge(right_keys, on=by_cols, how="left", indicator=True)
    unmatched_keys = unmatched_keys[unmatched_keys["_merge"] == "left_only"][by_cols]

    for _, key_row in unmatched_keys.iterrows():
        left_mask = pd.Series(True, index=left_df.index)

```

```

left_mask = pd.Series(True, index=left_df.index)
for c in by_cols:
    left_mask &= (left_df[c] == key_row[c])

left_sub = left_df[left_mask].copy()
for c in right_keep_cols:
    left_sub[c] = np.nan
left_sub[right_time_col] = pd.NaT
merged_parts.append(left_sub)

if len(merged_parts) == 0:
    return left_df.copy()

out = pd.concat(merged_parts, ignore_index=True)
return out

# -----
# Merge recent form
# -----
shots_with_form = grouped_asof_merge(
    left_df=shots_enriched,
    right_df=form_join,
    left_time_col="match_date",
    right_time_col="Date",
    by_cols=["season_code", "team_join"],
    right_keep_cols=["recent_form_5"],
    direction="backward",
    allow_exact_matches=False
)

print("shots_with_form shape:", shots_with_form.shape)
display(shots_with_form[[
    "match_id", "player_name", "team_name", "season_code",
    "team_join", "match_date", "recent_form_5"
]].head(10))

# -----
# Merge league position
# -----
shots_with_context = grouped_asof_merge(
    left_df=shots_with_form,
    right_df=standings_join,
    left_time_col="match_date",
    right_time_col="Date",
    by_cols=["season_code", "team_join"],
    right_keep_cols=["league_position", "cum_points", "cum_gd"],
    direction="backward",
    allow_exact_matches=False
)

print("shots_with_context shape:", shots_with_context.shape)
display(shots_with_context[[
    "match_id", "player_name", "team_name", "season_code",
    "team_join", "match_date", "recent_form_5",
    "league_position", "cum_points", "cum_gd"
]].head(10))

```

shots_with_form shape: (10837, 28)

	match_id	player_name	team_name	season_code	team_join	match_date	recent_form_5
0	3754141	Mesut Özil	Arsenal	NaN	NaN	2015-08-09	NaN
1	3754141	Alexis Alejandro Sánchez Sánchez	Arsenal	NaN	NaN	2015-08-09	NaN
2	3754141	Alexis Alejandro Sánchez Sánchez	Arsenal	NaN	NaN	2015-08-09	NaN
3	3754141	Olivier Giroud	Arsenal	NaN	NaN	2015-08-09	NaN
4	3754141	Santiago Cazorla González	Arsenal	NaN	NaN	2015-08-09	NaN
5	3754141	Alex Oxlade-Chamberlain	Arsenal	NaN	NaN	2015-08-09	NaN
6	3754141	Theo Walcott	Arsenal	NaN	NaN	2015-08-09	NaN
7	3754141	Olivier Giroud	Arsenal	NaN	NaN	2015-08-09	NaN
8	3754141	Theo Walcott	Arsenal	NaN	NaN	2015-08-09	NaN
9	3754141	Olivier Giroud	Arsenal	NaN	NaN	2015-08-09	NaN

shots_with_context shape: (2839, 31)

	match_id	player_name	team_name	season_code	team_join	match_date	recent_form_5	league_position	cum_points	cum_goals
0	3749493	Sylvain Wiltord	Arsenal	0304	Arsenal	2003-08-16	NaN	NaN	NaN	NaN
1	3749493	Thierry Henry	Arsenal	0304	Arsenal	2003-08-16	NaN	NaN	NaN	NaN
2	3749493	Thierry Henry	Arsenal	0304	Arsenal	2003-08-16	NaN	NaN	NaN	NaN
3	3749493	Kolo Habib Touré	Arsenal	0304	Arsenal	2003-08-16	NaN	NaN	NaN	NaN
4	3749493	Robert Pirès	Arsenal	0304	Arsenal	2003-08-16	NaN	NaN	NaN	NaN
5	3749493	Thierry Henry	Arsenal	0304	Arsenal	2003-08-16	NaN	NaN	NaN	NaN
6	3749493	Robert Pirès	Arsenal	0304	Arsenal	2003-08-16	NaN	NaN	NaN	NaN

```
# -----
# Cell 18: Compute score difference before each shot
# -----
goal_events = shots[shots["shot_outcome_name"] == "Goal"].copy()
goal_events = goal_events[["match_id", "index", "team_name"]].copy()

home_away_lookup = matches_flat[["match_id", "home_team_name", "away_team_name"]].drop_duplicates()

goal_events = goal_events.merge(home_away_lookup, on="match_id", how="left")
goal_events["home_goal"] = (goal_events["team_name"] == goal_events["home_team_name"]).astype(int)
goal_events["away_goal"] = (goal_events["team_name"] == goal_events["away_team_name"]).astype(int)

goal_events = goal_events.sort_values(["match_id", "index"]).copy()
goal_events["cum_home_goals"] = goal_events.groupby("match_id")["home_goal"].cumsum()
goal_events["cum_away_goals"] = goal_events.groupby("match_id")["away_goal"].cumsum()

shots_score = shots_with_context.merge(
    home_away_lookup,
    on="match_id",
    how="left",
    suffixes=("", "_lookup")
)

score_diffs = []

for _, s in shots_score.iterrows():
    prior_goals = goal_events[
        (goal_events["match_id"] == s["match_id"]) &
        (goal_events["index"] < s["index"])
    ]

    if len(prior_goals) == 0:
        home_goals = 0
        away_goals = 0
    else:
        last_goal_state = prior_goals.iloc[-1]
        home_goals = int(last_goal_state["cum_home_goals"])
        away_goals = int(last_goal_state["cum_away_goals"])

    if s["team_name"] == s["home_team_name"]:
        score_diff = home_goals - away_goals
    else:
```

```

score_diff = away_goals - home_goals

score_diffs.append(score_diff)

shots_score["score_diff_before_shot"] = score_diffs

display(shots_score[[
    "match_id", "player_name", "team_name", "index",
    "score_diff_before_shot"
]].head(10))

```

	match_id	player_name	team_name	index	score_diff_before_shot	
0	3749493	Sylvain Wiltord	Arsenal	179	0	
1	3749493	Thierry Henry	Arsenal	375	0	
2	3749493	Thierry Henry	Arsenal	520	0	
3	3749493	Kolo Habib Touré	Arsenal	607	0	
4	3749493	Robert Pirès	Arsenal	996	0	
5	3749493	Thierry Henry	Arsenal	1081	0	
6	3749493	Robert Pirès	Arsenal	1196	1	
7	3749493	Martin Keown	Arsenal	1716	1	
8	3749493	Patrick Vieira	Arsenal	1785	1	
9	3749493	Patrick Vieira	Arsenal	1970	1	

```

# -----
# Cell 19: Context factor and decision cost
# -----
def minute_pressure(minute):
    if pd.isna(minute):
        return 1.00
    if minute < 30:
        return 1.00
    elif minute < 60:
        return 1.05
    elif minute < 75:
        return 1.10
    else:
        return 1.20

def score_pressure(score_diff):
    if pd.isna(score_diff):
        return 1.00
    if score_diff < 0:
        return 1.12 # trailing
    elif score_diff == 0:
        return 1.08 # drawing
    else:
        return 0.98 # leading

def ranking_pressure(pos):
    if pd.isna(pos):
        return 1.00
    if pos <= 4:
        return 1.05
    elif pos >= 17:
        return 1.08
    else:
        return 1.00

def form_pressure(form5):
    if pd.isna(form5):
        return 1.00
    if form5 <= 3:
        return 1.06
    elif form5 >= 10:
        return 0.98
    else:
        return 1.00

final_shots = shots_score.copy()

final_shots["minute_factor"] = final_shots["minute"].apply(minute_pressure)
final_shots["score_factor"] = final_shots["score_diff_before_shot"].apply(score_pressure)
final_shots["ranking_factor"] = final_shots["league_position"].apply(ranking_pressure)
final_shots["form_factor"] = final_shots["recent_form_5"].apply(form_pressure)

```

```

final_shots["context_factor"] = (
    final_shots["minute_factor"] *
    final_shots["score_factor"] *
    final_shots["ranking_factor"] *
    final_shots["form_factor"]
)

final_shots["decision_cost"] = final_shots["delta"] * final_shots["context_factor"]

print("final_shots shape:", final_shots.shape)
display(final_shots[[
    "player_name", "team_name", "minute", "zone",
    "shot_xg", "xG_pass", "delta",
    "recent_form_5", "league_position", "score_diff_before_shot",
    "context_factor", "decision_cost"
]].head(10))

```

final_shots shape: (2839, 40)

	player_name	team_name	minute	zone	shot_xg	xG_pass	delta	recent_form_5	league_position	score_diff_before_s
0	Sylvain Wiltord	Arsenal	3	Z3_1	0.028333	0.105515	0.077182	NaN	NaN	
1	Thierry Henry	Arsenal	10	Z3_0	0.005752	0.084153	0.078401	NaN	NaN	
2	Thierry Henry	Arsenal	15	Z3_2	0.085029	0.111473	0.026444	NaN	NaN	
3	Kolo Habib Touré	Arsenal	18	Z3_2	0.084167	0.111473	0.027307	NaN	NaN	
4	Robert Pirès	Arsenal	31	Z3_2	0.031619	0.111473	0.079854	NaN	NaN	
5	Thierry Henry	Arsenal	34	Z3_1	0.783500	0.105515	-0.677985	NaN	NaN	
6	Robert Pirès	Arsenal	37	Z3_1	0.106324	0.105515	-0.000809	NaN	NaN	
7	Martin Keown	Arsenal	49	Z3_2	0.075699	0.111473	0.035775	NaN	NaN	
8	Patrick Vieira	Arsenal	51	Z3_2	0.132282	0.111473	-0.020809	NaN	NaN	
9	Patrick Vieira	Arsenal	57	Z3_1	0.025049	0.105515	0.080467	NaN	NaN	

```

# -----
# Cell 20: Final analytical dataset
# One row = one shot decision
# -----
final_dataset = final_shots[[
    "season_name", "season_code", "match_id", "match_date",
    "home_team_name", "away_team_name",
    "team_name", "player_name",
    "minute", "second", "index",
    "x", "y", "zone",
    "shot_xg", "xG_pass", "zone_pass_shot_samples", "delta",
    "recent_form_5", "league_position", "cum_points", "cum_gd",
    "score_diff_before_shot",
    "minute_factor", "score_factor", "ranking_factor", "form_factor",
    "context_factor", "decision_cost"
]].copy()

print("final_dataset shape:", final_dataset.shape)
display(final_dataset.head(10))

```